



Chương 11 Multi-Agent & Agent Orchestration

Khi một AI không đủ - Xây đội ngũ thay vì siêu nhân

Playbook SDD + ADD | Bản trình chiếu chuyên sâu cho chuyên gia công nghệ về kiến trúc multi-agent, skill system, hooks automation, MCP orchestration và chiến lược vận hành AI team ở mức production.

Từ Single Agent sang AI Team

Cho đến trước chương này, workflow chuẩn là một agent duy nhất đọc spec, lập kế hoạch, viết code, chạy test và tự sửa lỗi. Cách làm đó rất hiệu quả cho feature nhỏ hoặc bounded context ngắn.

Khi dự án lớn hơn, cần frontend, backend, testing chạy đồng thời và context của một feature vượt giới hạn cửa sổ nhớ, mô hình single-agent bắt đầu bộc lộ giới hạn kiến trúc. Vấn đề không phải agent kém thông minh, mà do bài toán thực tế đòi hỏi chuyên môn hóa và song song hóa.

Kỹ sư giỏi trong tương lai không phải là người code nhanh nhất, mà là người biết tổ chức đội ngũ AI hiệu quả nhất: phân việc đúng, resolve conflict đúng thời điểm và đảm bảo output đi đúng hướng sản phẩm.

Cấu trúc Chương 11

1

11.1 Tại sao 1 Agent không đủ?

Context limits, cognitive load, parallel execution.

2

11.2 Kiến trúc Multi-Agent

Vertical vs horizontal, shared state, communication patterns.

3

11.3 Skill System

SKILL.md format, 10 must-have skills, custom skill workflow.

4

11.4 Hooks & Automation

Lifecycle hooks, pre-commit, self-healing loop.

5

11.5 Hands-on Lab

4-agent team export report to PDF, contract-first execution.

6

11.6 MCP trong Multi-Agent

Tool borrowing, per-agent permissions, tool-sharing protocol.

11.1

Tại sao 1 Agent không đủ?

Giới hạn kiến trúc cơ bản, không biến mất kể cả khi model mạnh hơn



11.1.1

Context Window Limits

Context window dù 200K hay 2M tokens vẫn là tài nguyên hữu hạn. Một codebase production thường gồm frontend, backend, DB schema, tests và specs; khi dồn mọi thứ vào cùng context, không gian cho reasoning chất lượng cao bị co lại rất nhanh.

Điểm quan trọng là scale context không giải quyết triệt để. Khi enterprise codebase tăng lên hàng trăm nghìn dòng, lượng tokens tăng nhanh hơn khả năng giữ tập trung của một agent duy nhất.

Giải pháp thực dụng: không cần một agent biết tất cả. Frontend agent giữ frontend + shared types; backend agent giữ backend + DB schema; lead agent giữ interface contracts và orchestration.

Codebase vs Context Window

63K

Lines trong monorepo cỡ vừa

945K

Tokens ước tính (15 tokens/line)

4.7x

Lớn hơn context 200K tokens

Thư mục	Lines
frontend/src/	~15,000 lines
backend/src/	~25,000 lines
shared/types/	~3,000 lines
tests/	~12,000 lines
docs/specs/	~8,000 lines

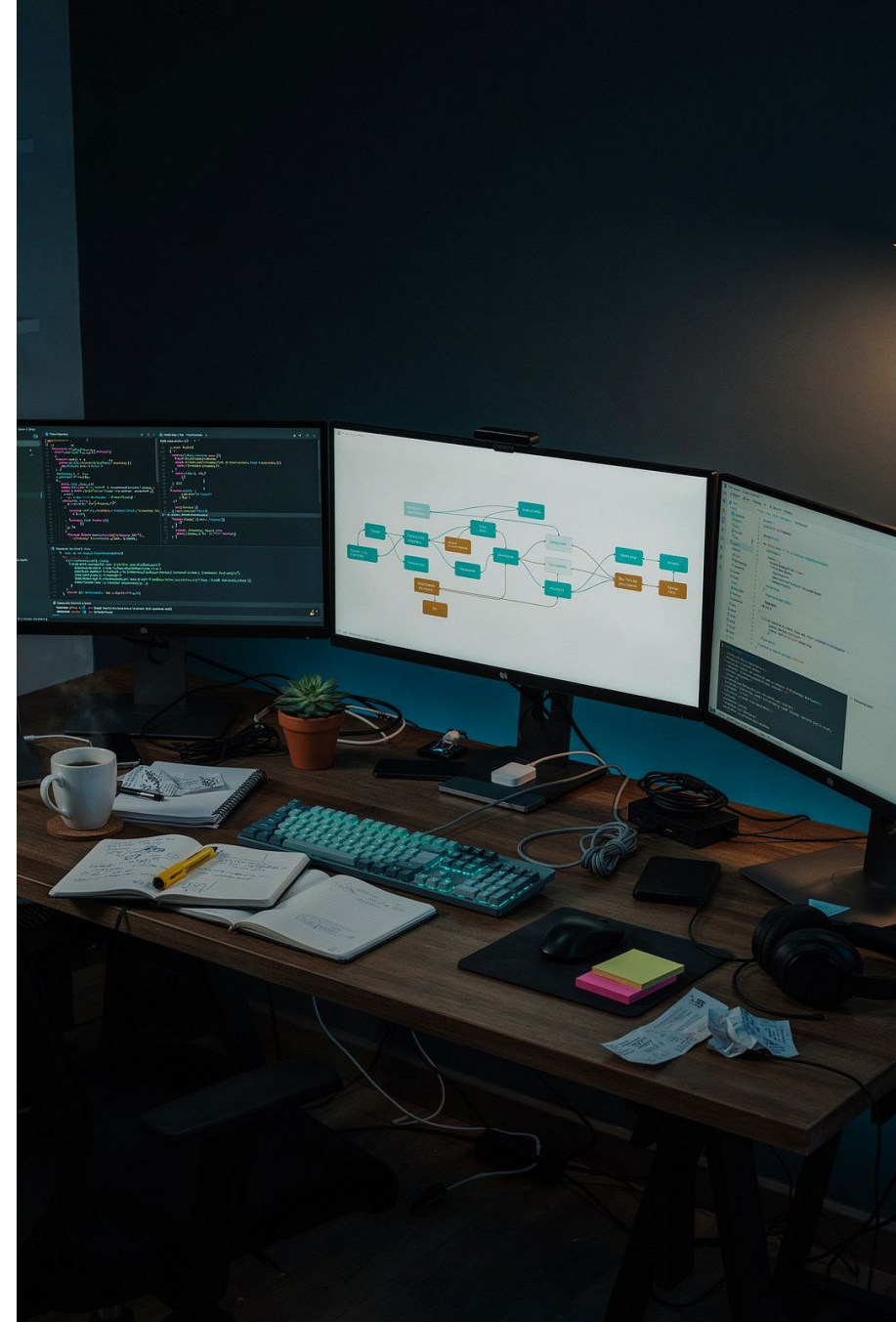
Enterprise scale: 500K+ lines ~7.5M tokens - vẫn không fit kể cả context 1M.

11.1.2

Cognitive Load của AI

Khi một model phải track quá nhiều concerns cùng lúc, attention bị phân tán và chất lượng suy luận theo từng concern giảm. Agent có thể làm nhiều thứ, nhưng làm đồng thời CSS animation, SQL optimization, business logic và test strategy trong một session thường dẫn đến output "đủ dùng" chứ không xuất sắc.

Chuyên môn hóa không phải vì agent thiếu intelligence; đó là cơ chế để dồn attention vào một domain, giữ best-practice context sâu và giảm rework ở integration phase.



Scenario vs Quality

Scenario	Agent phải track	Reasoning quality	Output quality
Full-stack trong 1 session	CSS + SQL + business logic + tests + docs	Phân tán, ~60%	Inconsistent, cần rework nhiều
Frontend-only agent	UI patterns, state, a11y, CSS	Focused, ~90%	High quality, rework thấp
Backend-only agent	DB schema, rules, API contracts	Focused, ~90%	High quality, rework thấp
Test-only agent	Coverage criteria, mocking, edge cases	Focused, ~95%	Thorough, bắt lỗi tốt hơn

11.1.3

Parallel Execution tối ưu thời gian chờ

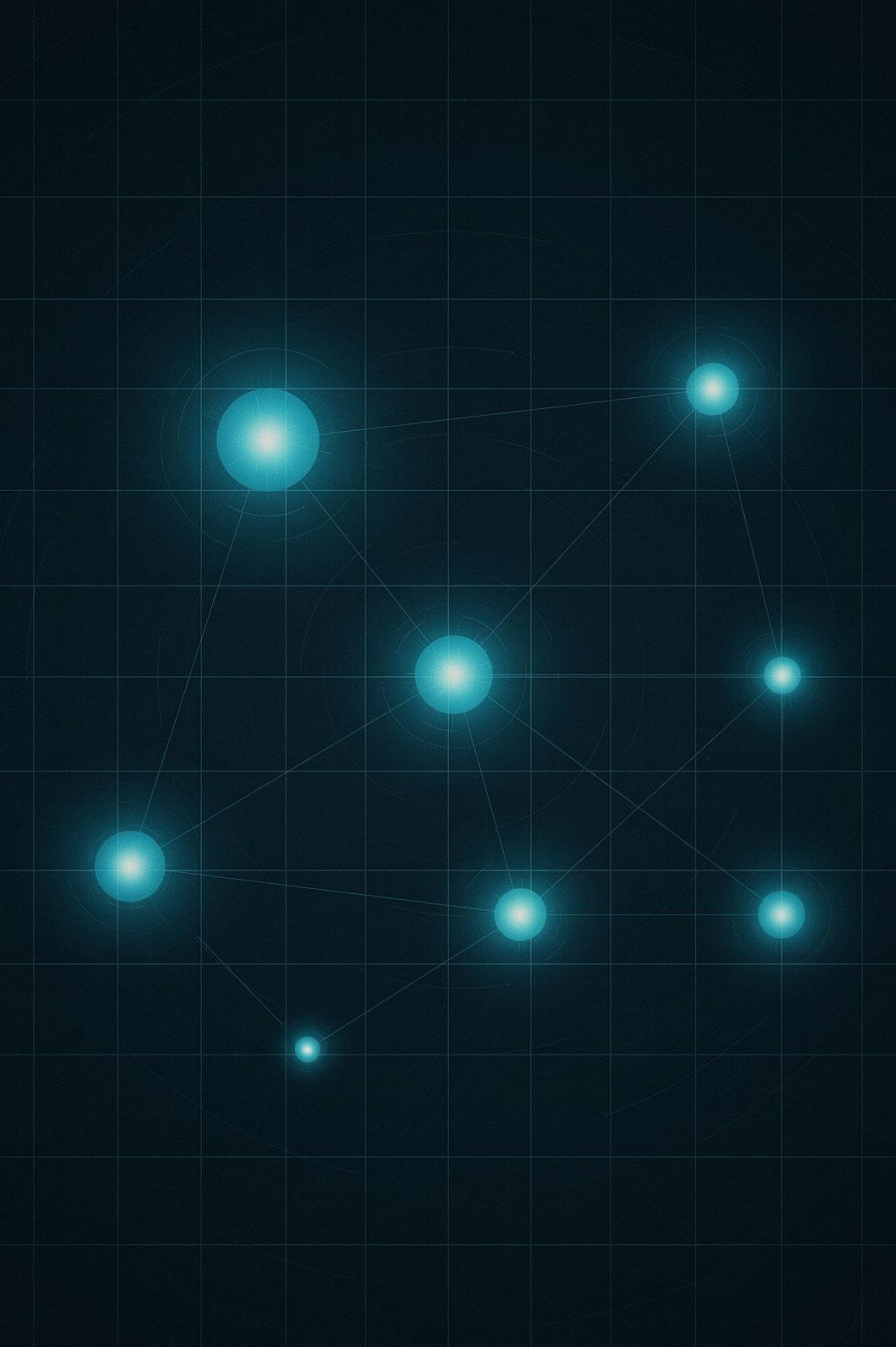
Nhiều task trong một feature cycle độc lập tự nhiên: backend API, frontend scaffolding, docs hoặc worker service có thể triển khai song song ngay sau khi contract được define. Đây là lợi ích trực tiếp lên developer experience vì giảm idle time.

Sequential

1 agent chạy tuần tự backend 4h -> frontend 4h -> tests 3h -> docs 2h -> fix 3h, tổng khoảng 16h.

Parallel

3 agents chạy đồng thời ngày 1, ngày 2 chỉ còn phase integration ngắn; tổng còn khoảng 7h, giảm ~56% thời gian elapsed.



11.2

Kiến trúc Multi-Agent

Thiết kế đúng pattern, rõ trade-off, rõ điểm nghẽn

11.2.1

Vertical vs Horizontal

Vertical (Lead + Workers)

- Lead nhận intent cấp cao, plan và delegate.
- Trách nhiệm rõ, dễ audit, conflict đơn giản.
- Nhược điểm: lead có thể thành bottleneck/SPOF.

Horizontal (Peer Agents)

- Agents ngang hàng sync qua shared context.
- Không bottleneck trung tâm, parallelism tốt.
- Nhược điểm: conflict resolution phức tạp hơn.

Chọn kiến trúc theo bounded context: team feature nhỏ thường phù hợp vertical; workstreams độc lập mạnh thường hợp horizontal.

Shared State & Context Synchronization

Thất bại phổ biến nhất của multi-agent không nằm ở chất lượng từng agent, mà nằm ở communication drift. Khi backend đổi field hoặc schema mà frontend không biết ngay, integration sẽ đổ vỡ vì assumptions lệch nhau.

shared_context.md phải là single source of truth: Lead cập nhật sau mỗi major decision; mọi sub-agent bắt buộc đọc trước khi bắt đầu và trước mỗi thay đổi liên quan API/shared files.

```
# .sdd/shared_context.md (backbone)
## LAST UPDATED: 2025-01-20 15:30 UTC
## API CONTRACTS
POST /auth/register -> { user_id, created_at }
POST /auth/login    -> { access_token, refresh_token }
GET  /orders        -> { orders, meta } # renamed from order_list
## KNOWN BREAKING CHANGES
2025-01-20: order_list -> orders
Impact: Frontend parser must update
```

POST /auth/register -> Response { user_id, created_at } GET /orders -> Response { orders, meta } Known breaking change: "order_list" -> "orders" Impact: Frontend parser phải update ngay.

11.2.2

Lead Agent Conflict Resolution

Xung đột thường xảy ra trên shared resources như `api_contracts.yaml`, DB schema hoặc config hệ thống. Protocol đúng là mọi write vào shared artifacts phải qua approval của Lead Agent.

Ví dụ conflict tương thích: Backend muốn rename `image_path` -> `image_url`, Frontend muốn thêm `imageUrl`. Lead quyết định thứ tự thực thi để tránh trạng thái trung gian sai: backend rename trước, frontend cập nhật theo sau.

Communication Patterns

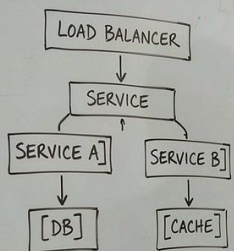
Pattern	Cơ chế	Use case	Ví dụ
File-based handoff	Agent A ghi file, B đọc tiếp	Pipeline tuần tự	Backend viết API spec -> Frontend đọc
Shared database/file	Cùng đọc/ghi shared state	Real-time sync	shared_context.md, SQLite
Event queue	Publish/subscribe events	Loose coupling	Backend done -> Frontend notified
Direct delegation	Lead spawn sub-agent	Orchestrated execution	Claude Code multi-agent
Spec-based contract	Coordinate qua SPEC.md	Ngừa conflict từ sớm	SDD workflow

Nguyên tắc thiết kế Agent Team

Mỗi agent cần domain rõ ràng, overlap thấp, coupling giao tiếp tối thiểu và có khả năng fail/restart độc lập. Không phải dự án nào cũng cần tăng số lượng agent, vì orchestration overhead cũng là chi phí thật.

Team size	Khuyến nghị team agent	Khi phù hợp
1 developer	1 generalist agent	Prototype, personal project
2-3 developers	1 Lead + 2 specialists (FE + BE)	Small product team
4-6 developers	1 Lead + 3-4 specialists	Nhiều modules/services
7+ developers	1 Lead + domain teams có sub-lead	Large product, nhiều squads

SYSTEM ARCHITECTURE v2.4 — Q3 REVIEW



LATENCY: 120ms
THROUGHPUT: 2.4k/s
ERROR RATE: 0.03%

API GATEWAY

- AUTH
- ROUTING
- RATE LIMIT

CHECKLIST:

- ☒ Code review
- ☒ Unit tests
- ☐ Integration
- ☐ Deploy
- ☐ Monitor

TODO:

- Scale DB
- Add caching
- Review auth

11.3

Skill System

Đóng gói kinh nghiệm senior thành instruction set cho agent

11.3

Vì sao SKILL.md có giá trị chiến lược?

Trong AI-assisted development, knowledge transfer là điểm nghẽn lớn: conventions, performance patterns, security gotchas và domain idioms thường nằm trong đầu senior chứ không nằm trong README. SKILL.md biến kiến thức ngầm đó thành quy tắc thực thi được.

SKILL.md không phải tutorial để con người đọc; nó là playbook vận hành để agent làm đúng chuẩn senior ngay cả khi người chạy agent còn mới với codebase.

11.3.1

SKILL.md Format chuẩn

Một skill tối thiểu cần metadata, role rõ, workflow từng bước, patterns/anti-patterns và checklist tự kiểm tra. Cấu trúc này buộc knowledge phải cụ thể thay vì chung chung.

name: [Skill Name]

version: 1.0.0

author: @senior-dev

domain: [backend | frontend | testing | database | security | devops]

tools: [Cline, Cursor, Claude Code]

trigger: [Khi nào agent apply skill]

ROLE -> EXPERTISE -> WORKFLOW -> PATTERNS -> ANTI-PATTERNS -> CHECKLIST

11.3.2

Skill 1: SQL Performance Tuner

Skill này xử lý pain point lớn ở backend data layer: query chậm, index sai thứ tự, pagination kém hiệu quả và N+1. Agent buộc phải đi qua quy trình phân tích access pattern, cardinality và kiểm chứng bằng EXPLAIN ANALYZE.

Patterns

- Ưu tiên composite index theo query thực tế.
- Tránh `SELECT *` trong production path.
- Dùng cursor-based pagination cho bảng lớn.

Anti-patterns

- Anti-pattern: N+1 query trong loop.
- Anti-pattern: non-sargable predicate.
- Anti-pattern: thiếu index cho foreign keys.

11.3.2

Skill 2: API Security Auditor

Mục tiêu của skill này là chuẩn hóa kiểm tra OWASP Top 10 ngay ở lúc implement endpoint, không đợi đến audit muộn. Checklist bắt buộc gồm authentication, authorization, input validation, rate limiting, error leakage, SQL injection và logging dữ liệu nhạy cảm.

Lỗi kinh điển cần chặn: đồng nhất authentication với authorization, leak khác biệt `not found`/`forbidden`, và log request body chưa sanitize.

Skills 3-10 và Trigger

Skill	Domain	Vấn đề giải quyết	Trigger
3. React Component Architect	Frontend	Component design, state, rerender optimization	Khi viết React components
4. Go Error Handler	Backend	Error wrapping, sentinel errors, logging	Khi handle errors trong Go
5. Test Coverage Engineer	Testing	Boundary values, mock design, strategy	Khi viết test files
6. Docker/K8s Deployer	DevOps	Build stages, limits, health checks	Khi viết Dockerfile/manifests
7. Async Task Designer	Backend	Queue patterns, retry, idempotency	Khi build background jobs
8. Frontend Performance	Frontend	Bundle size, lazy loading, CWV	Khi optimize performance
9. Data Migration Safe	Database	Zero-downtime migration, rollback	Khi viết DB migrations
10. OpenAPI Documenter	Documentation	Schema consistency, examples, errors	Khi document API

11.3.2

Skill 8: Frontend Performance Optimizer

Skill mẫu hoàn chỉnh đặt target cụ thể cho Core Web Vitals: LCP < 2.5s, FID < 100ms, CLS < 0.1 và bundle per route < 150KB gzipped. Nhờ target định lượng, agent có tiêu chuẩn pass/fail rõ.

Patterns

- Dynamic import cho component nặng
- Dùng Next.js Image thay vì img thường
- useMemo/useCallback để tránh re-render thừa

Anti-patterns

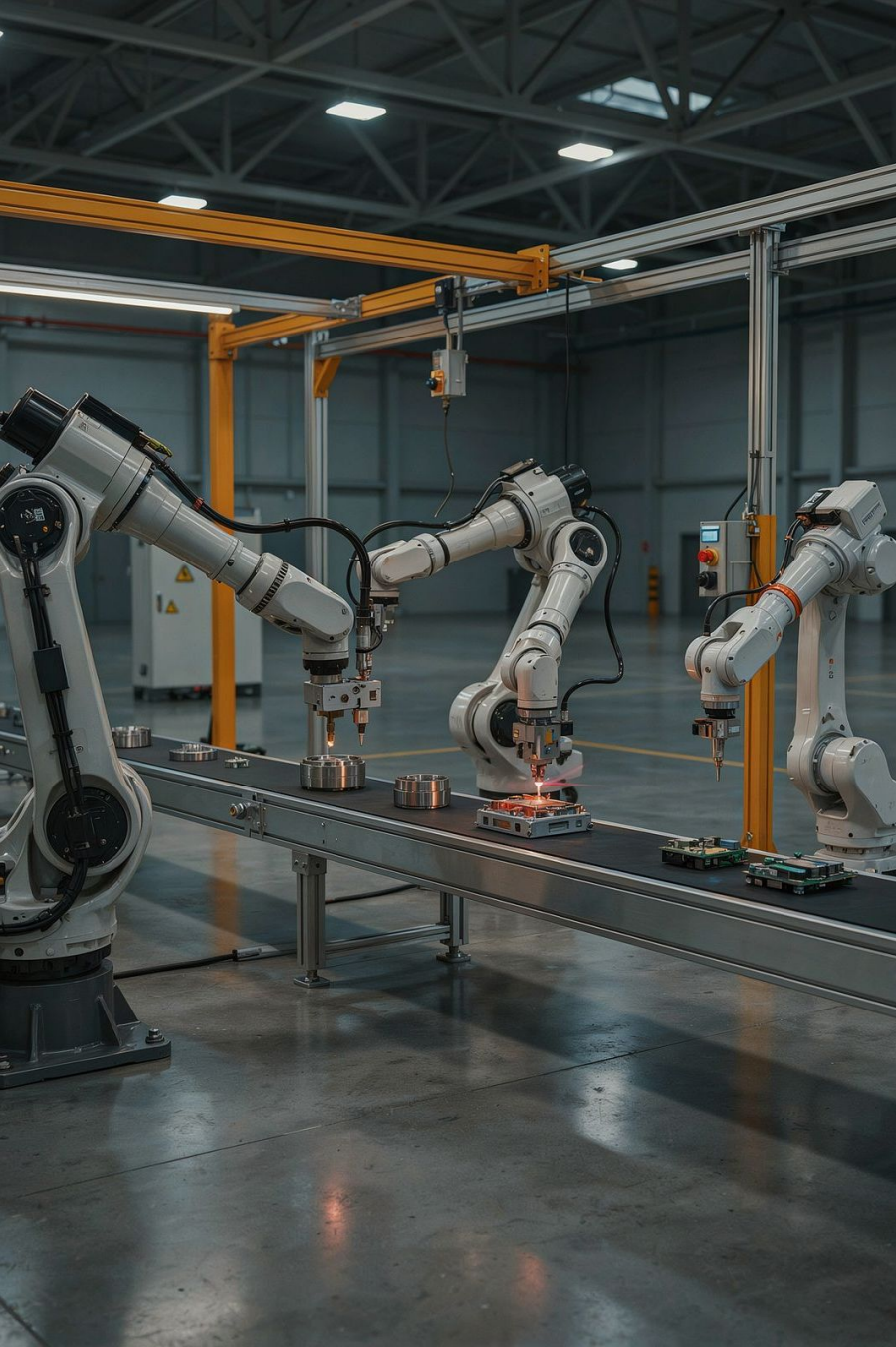
- useEffect không deps array
- Inline object/function props không memo
- import toàn bộ lodash

11.3.3

Custom Skill cho dự án

Custom skills thường cho ROI cao nhất vì chứa domain knowledge riêng của công ty. Workflow chuẩn: xác định pain point lặp lại, phỏng vấn senior để lấy pattern thật, viết draft theo template, test với agent và iterate 2-3 vòng đến khi stable.

Ví dụ Payment Safety Guard (fintech): bắt buộc idempotency key ở mọi payment call, verify webhook HMAC, chặn concurrent attempts cho cùng order, và đảm bảo amount invariant sau order creation để tránh double-charge.



11.4

Hooks và Automation

Tạo Self-Healing Loop để hệ thống tự kiểm soát chất lượng

Lifecycle Hooks

Hook	Timing	Use case	Ví dụ action
pre_file_edit	Trước khi edit file	Backup, validation	Create checkpoint, validate constraints
post_file_edit	Sau khi edit file	Auto-format, lint	Run gofmt/prettier/eslint --fix
pre_test	Trước khi chạy tests	Environment setup	Start test DB, seed fixtures
post_test	Sau khi chạy tests	Report, cleanup	Coverage report, cleanup DB
pre_commit	Trước git commit	Quality gate	Run tests, lint, secret checks
post_commit	Sau commit	Notification, sync	Update plan.md, notify lead agent

Pre-commit Hook: Code luôn sạch

Pre-commit đóng vai trò quality gate cho AI-generated code: format, lint, type-check, secret scan và unit tests trước khi ghi lịch sử Git. Mục tiêu là chuyển effort review của con người sang logic và kiến trúc thay vì style lỗi vặt.

Flow chuẩn: gofmt/prettier -> golangci-lint -> go vet -> gitleaks -> go test -tags=unit Nếu bước nào fail: chặn commit ngay. Nếu auto-format có thay đổi: git add -u để re-stage.

```
# .git/hooks/pre-commit (rút gọn)
set -e
gofmt -w $(find . -name "*.go" -not -path "./vendor/*")
golangci-lint run --timeout=60s
go vet ./...
gitleaks detect --staged --no-banner || true
go test ./... -tags=unit -timeout=30s
git add -u
```

Self-Healing Loop

Self-healing loop vận hành theo chu trình: hook phát hiện test fail, thu log lỗi, gọi fixer agent với constraints rõ, re-run tests và chỉ cho commit đi tiếp khi pass. Nếu vượt ngưỡng số lần thử thì dừng và escalate cho human review.

3

Max attempts mặc định

PASS

Cho phép commit tự động

FAIL

Notify human, manual review

Boundary bắt buộc: chỉ auto-heal lỗi predictable, low-risk như format/lint/simple assertions. Security issues hoặc architecture violations phải để con người quyết định.

```
# scripts/self_heal.sh (rút gọn)
MAX_ATTEMPTS=3
ATTEMPT=1
while [ $ATTEMPT -le $MAX_ATTEMPTS ]; do
  cline --no-interactive --task "Tests are failing. Fix implementation only."
  go test ./... -tags=unit -timeout=30s && exit 0
  ATTEMPT=$((ATTEMPT + 1))
done
echo "Manual review required" && exit 1
```

11.4.4

Claude Code Hook Configuration

Cấu hình hooks trong `.claude/settings.json` cho phép chèn automation vào PreToolUse/PostToolUse: checkpoint trước khi write/edit, auto-format theo ngôn ngữ sau khi write và tự tạo coverage report sau các lệnh test.

PreToolUse

matcher Write|Edit -> append git checkpoint log

PostToolUse: Go

Write.*\..go\$ -> gofmt + go vet

PostToolUse: TypeScript

Write.*\.(ts|tsx)\$ -> prettier --write

PostToolUse: Test

Bash.*go test -> generate coverage.html



11.5

Hands-on Lab: Export Report sang PDF

Contract-first, 3 agents chạy đồng thời, lead orchestrates integration

11.5.1

Setup team 4 agents

Lab chọn feature Export Report to PDF vì tách tự nhiên thành ba concerns độc lập: UI/UX, business logic và worker xử lý PDF + storage. Đây là điều kiện lý tưởng để thực hành multi-agent orchestration thực chiến.

Lead Agent: orchestration, conflict resolution, ownership `shared_context.md`.

Agent A/B/C: UI, Logic, Worker chạy ở các session riêng với context giới hạn đúng domain.

```
# Lab setup — 4-agent team
# LEAD: orchestrates, resolves conflicts, owns shared_context.md
# AGENT A: UI (button/modal/loading/error)
# AGENT B: Business logic (DB query, transform, validation)
# AGENT C: Worker (PDF rendering, S3 upload, async processing)
# Claude Code: claude --mode=multi-agent --agents=4 --orchestrator=lead
```

Step 1: Interface Contract do Lead định nghĩa

Lead phải khóa contract trước khi giao việc để sub-agents có thể làm độc lập mà không tạo coupling ngầm. Contract gồm API giữa UI-Backend, job structure giữa Logic-Worker và shared types dùng chung TypeScript/Go.

```
# Lead Agent Prompt
"Chúng ta implement Export Report to PDF.
Define contracts cho 3 sub-agents:
1) API contract 2) Job contract 3) Shared types
Update .sdd/shared_context.md.
Sub-agents bắt buộc đọc file này trước khi bắt đầu."
```

```
POST /api/reports/export
Request: { report_type, date_range, format:"pdf" }
Response: { job_id, status_url }

GET /api/reports/jobs/:id
Response: { status: pending|processing|done|failed, download_url?, error? }

ExportJob: { id, report_data, metadata, output_path }
```

Step 2: Parallel Execution

Sau khi contract rõ, Agent A xây UI polling mỗi 3 giây và xử lý loading/error/success; Agent B dựng handlers + query DB + enqueue Kafka; Agent C dựng Kafka consumer, render PDF A4, upload S3 và cập nhật job status.

Agent A Prompt (UI)

Đọc .sdd/shared_context.md. Implement Export PDF button + polling 3s.

DoD: render đúng, polling stop khi **done**/failed, no TS errors.

Agent B Prompt (Logic)

Implement POST /api/reports/export + GET /api/reports/jobs/:**id**.

Query DB, enqueue Kafka topic "**export-jobs**", update shared_context status.

Agent C Prompt (Worker)

Consume "**export-jobs**", render PDF A4, upload S3 exports-**{env}**,

update status processing -> **done**/failed.

Mỗi agent chỉ cần biết contract và phạm vi domain của mình. Đây là chìa khóa để đạt parallelism thật mà không tạo đồng merge conflict khó kiểm soát.

Step 3: Lead Orchestrates & Merges

Khi A/B/C báo DONE, lead làm integration check theo contract, chạy test tích hợp và xử lý mismatch tại điểm giao nhau. Conflict tiêu biểu trong lab là khác biệt naming convention giữa backend và worker (`report_data` vs `reportData`).

```
# Lead integration checks
cat frontend/src/hooks/useExportReport.ts | grep "api/reports"
cat backend/internal/handler/report_handler.go | grep "export"
make test-integration
# Nếu mismatch: assign fix task theo ownership, re-run tests.
```

Lead quyết định chuẩn thống nhất (snake_case theo Go convention), giao fix task đúng agent chịu trách nhiệm, rồi re-run integration tests đến khi pass toàn bộ.

"Multi-agent không loại bỏ conflict.
Nó dồn conflict về integration point,
nơi Lead Agent phải làm traffic controller."

11.6

MCP trong Multi-Agent

Hệ thần kinh chung để agents mượn capability có kiểm soát



11.6.1

Tool Borrowing

Trong team agent, MCP không chỉ để truy cập external resources mà còn để mượn capability liên-agent theo access policy. Ví dụ Frontend Agent cần verify UI bằng dữ liệu thật nhưng chỉ được DB read-only, không được write trực tiếp.

Key insight: cùng một cụm MCP servers nhưng permissions khác nhau theo từng agent (lead rộng quyền, frontend read-only DB, backend produce Kafka, worker consume Kafka + write S3).

Per-Agent Access Control

`.sdd/mcp-config.yaml` (rút gọn)

agents:

lead-agent: { database: read_write, github: [repo:read, issues:write], fs: [".sdd/", "src/", "tests/"] }

frontend-agent: { database: read_only, fs: ["frontend/src/", ".sdd/shared_context.md"] }

backend-agent: { database: read_write, kafka: produce(export-jobs), fs: ["backend/", ".sdd/"] }

worker-agent: { database: read_write, kafka: consume(export-jobs), s3: write(exports-dev,exports-staging), fs: ["worker/", ".sdd/"] }

lead-agent

database read_write, github
repo:read/issues:write,
filesystem .sdd+src+tests

frontend-agent

database read_only (public),
filesystem frontend/src +
shared_context

backend-agent

database read_write, kafka
produce(export-jobs),
filesystem backend + .sdd

worker-agent

database read_write, kafka
consume(export-jobs), s3
write (non-prod buckets)

Thiết kế này cho phép tách biệt trách nhiệm, giảm blast radius khi sai sót và hỗ trợ audit trail rõ ràng ở cấp quyền truy cập.

11.6.2

MCP như Tool-Sharing Protocol

Một mô hình advanced là agent publish MCP server để chia sẻ capability cho agent khác. Ví dụ Backend Agent expose `report-data` MCP server; Frontend/Worker gọi qua interface này thay vì đưng trực tiếp DB schema hoặc query logic nội bộ.

```
@server.tool()
async def get_report_data(report_type, from_date, to_date):
    data = await report_service.fetch(report_type, from_date, to_date)
    return data.to_dict()
# Frontend session gọi MCP server của Backend để preview/report validation.
```

Tổng kết Chương 11

Section	Key concept	Takeaway
11.1	Context limit + cognitive load + parallelism	Một agent tốt; nhiều agents chuyên hóa tốt hơn
11.2	Architecture + shared_context.md	Communication là vấn đề lớn nhất
11.3	SKILL.md encode kinh nghiệm senior	Build 10 must-have + custom skills theo domain
11.4	Hooks + self-healing loop	Pre-commit giữ sạch code, tự sửa lỗi predictable
11.5	Lab Export PDF 4 agents	Contract first -> parallel -> lead orchestration
11.6	MCP neural network cho team	Tool borrowing có access control, không chia sẻ vô hạn

📌 Tư duy cốt lõi: chuyển từ "lập trình viên bị AI thay thế" sang "lập trình viên quản lý AI team" để tăng giá trị deliver trên cùng đơn vị thời gian.